CHAPTER 9

Pointers

TOPICS

9.1 Getting the Address of a Variable
9.2 Pointer Variables
9.3 The Relationship between Arrays and Pointers
9.4 Pointer Arithmetic
9.5 Initializing Pointers
9.6 Comparing Pointers
9.7 Pointers as Function Parameters
9.8 Dynamic Memory Allocation
9.9 Returning Pointers from Functions
9.10 Using Smart Pointers to Avoid Memory Leaks
9.11 Focus on Problem Solving and Program Design: A Case Study

9.1 Getting the Address of a Variable

CONCEPT: The address operator (&) returns the memory address of a variable.

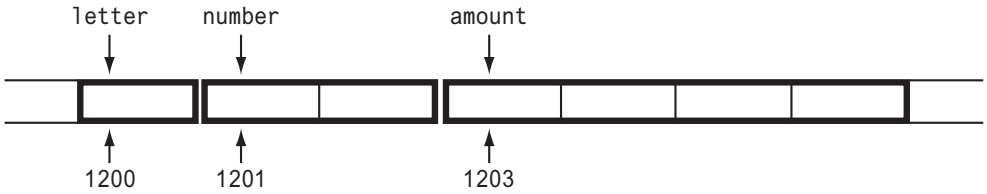
Every variable is allocated a section of memory large enough to hold a value of the variable's data type. On a PC, for instance, it's common for 1 byte to be allocated for `chars`, 2 bytes for `shorts`, 4 bytes for `ints`, `longs`, and `floats`, and 8 bytes for `doubles`.

Each byte of memory has a unique *address*. A variable's address is the address of the first byte allocated to that variable. Suppose the following variables are defined in a program:

```
char letter;
short number;
float amount;
```

Figure 9-1 illustrates how they might be arranged in memory and shows their addresses.

Figure 9-1 Variables and their addresses



In Figure 9-1, the variable `letter` is shown at address 1200, `number` is at address 1201, and `amount` is at address 1203.



NOTE: The addresses of the variables shown in Figure 9-1 are arbitrary values used only for illustration purposes.

Getting the address of a variable is accomplished with an operator in C++. When the address operator (`&`) is placed in front of a variable name, it returns the address of that variable. Here is an expression that returns the address of the variable `amount`:

```
&amount
```

And here is a statement that displays the variable's address on the screen:

```
cout << &amount;
```



NOTE: Do not confuse the address operator with the `&` symbol used when defining a reference variable.

Program 9-1 demonstrates the use of the address operator to display the address, size, and contents of a variable.

Program 9-1

```
1  // This program uses the & operator to determine a variable's
2  // address and the sizeof operator to determine its size.
3  #include <iostream>
4  using namespace std;
5
6  int main()
7  {
8      int x = 25;
9
10     cout << "The address of x is " << &x << endl;
11     cout << "The size of x is " << sizeof(x) << " bytes\n";
12     cout << "The value in x is " << x << endl;
13     return 0;
14 }
```

Program Output

```
The address of x is 0x8f05
The size of x is 4 bytes
The value in x is 25
```



NOTE: The address of the variable x is displayed in hexadecimal. This is the way addresses are normally shown in C++.

9.2 Pointer Variables

CONCEPT: *Pointer variables*, **which are often just called *pointers*, are designed to hold memory addresses. With pointer variables, you can indirectly manipulate data stored in other variables.**

A *pointer variable*, which often is just called a *pointer*, is a special variable that holds a memory address. Just as `int` variables are designed to hold integers, and `double` variables are designed to hold floating-point numbers, pointer variables are designed to hold memory addresses.

Memory addresses identify specific locations in the computer's memory. Because a pointer variable holds a memory address, it can be used to hold the location of some other piece of data. This should give you a clue as to why it is called a pointer: It "points" to some piece of data that is stored in the computer's memory. Pointer variables also allow you to work with the data that they point to.

We've already used memory addresses in this book to work with data. Recall from Chapter 6 that when we pass an array as an argument to a function, we are actually passing the array's beginning address. For example, suppose we have an array named `numbers`, and we call the `showValues` function as shown here:

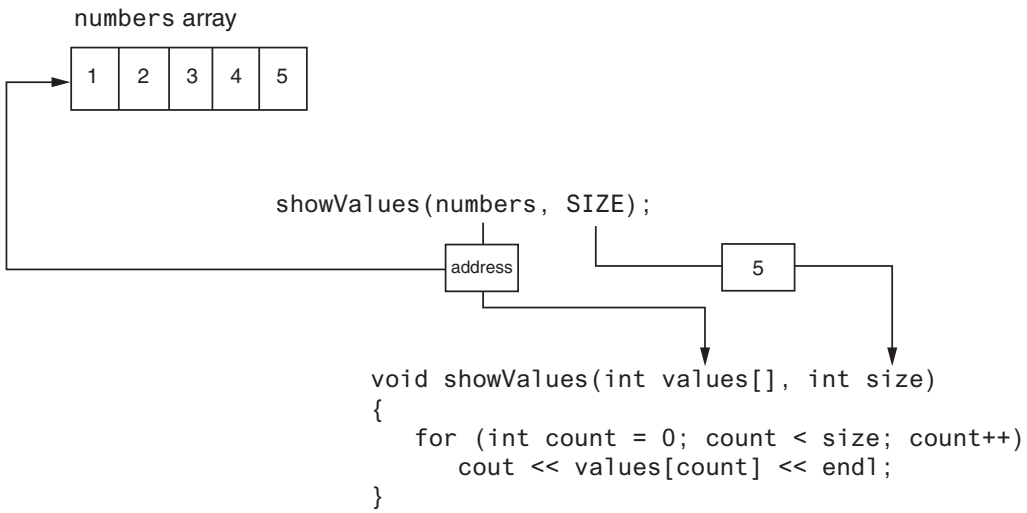
```
const int SIZE = 5;
int numbers[SIZE] = { 1, 2, 3, 4, 5 };
showValues(numbers, SIZE);
```

In this code, we are passing the name of the array, `numbers`, and its size as arguments to the `showValues` function. Here is the definition for the `showValues` function:

```
void showValues(int values[], int size)
{
   for (int count = 0; count < size; count++)
      cout << values[count] << endl;
}
```

In the function, the `values` parameter receives the address of the `numbers` array. It works like a pointer because it "points" to the `numbers` array, as shown in Figure 9-2.

Figure 9-2 Passing an array to a function



Inside the `showValues` function, anything done to the `values` parameter is actually done to the `numbers` array. We can say that the `values` parameter references the `numbers` array.

Also, recall from Chapter 6 that we discussed reference variables. A reference variable acts as an alias for another variable. It is called a reference variable because it references another variable in the program. Anything you do to the reference variable is actually done to the variable it references. For example, suppose we have the variable `jellyDonuts`, and we pass the variable to the `getOrder` function, as shown here:

```
int jellyDonuts;
getOrder(jellyDonuts);
```

Here is the definition for the `getOrder` function:

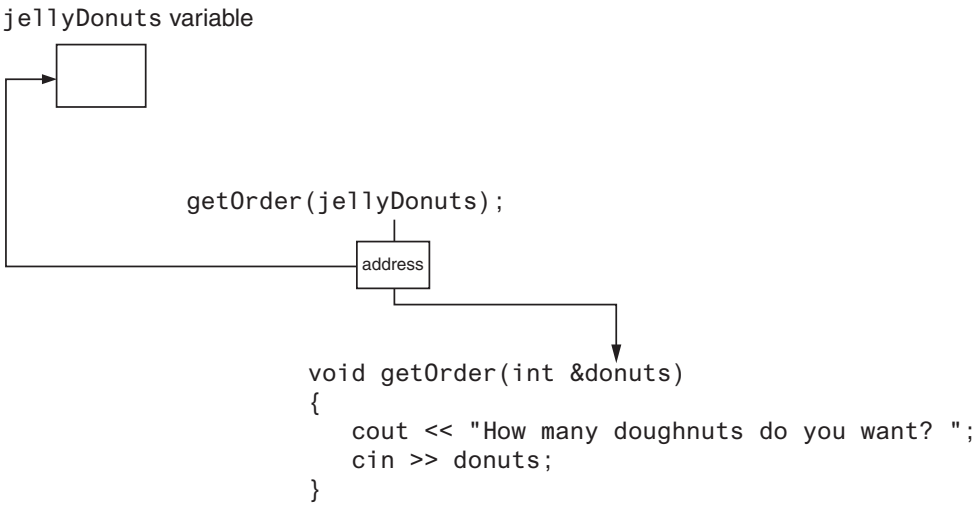
```
void getOrder(int &donuts)
{
   cout << "How many doughnuts do you want? ";
   cin >> donuts;
}
```

In the function, the `donuts` parameter is a reference variable, and it receives the address of the `jellyDonuts` variable. It works like a pointer because it "points" to the `jellyDonuts` variable as shown in Figure 9-3.

Inside the `getOrder` function, the `donuts` parameter references the `jellyDonuts` variable. Anything done to the `donuts` parameter is actually done to the `jellyDonuts` variable. When the user enters a value, the `cin` statement uses the `donuts` reference variable to indirectly store the value in the `jellyDonuts` variable.

Notice the connection between the `donuts` reference variable and the `jellyDonuts` argument is automatically established by C++ when the function is called. When you are writing this code, you don't have to go through the trouble of finding the memory address of the `jellyDonuts` variable then properly storing that address in the `donuts` reference variable.

Figure 9-3 Passing an argument by reference



When you are storing a value in the `donuts` variable, you don't have to specify that the value should actually be stored in the `jellyDonuts` variable. C++ handles all of that automatically.

In C++, pointer variables are yet another mechanism for using memory addresses to work with pieces of data. Pointer variables are similar to reference variables, but pointer variables operate at a lower level. By this, I mean that C++ does not automatically do as much work for you with pointer variables as it does with reference variables. In order to make a pointer variable reference another item in memory, you have to write code that fetches the memory address of that item and assigns the address to the pointer variable. Also, when you use a pointer variable to store a value in the memory location that the pointer references, your code has to specify that the value should be stored in the location referenced by the pointer variable, and not in the pointer variable itself.

Because reference variables are easier to work with, you might be wondering why you would ever use pointers at all. In C++, pointers are useful, and even necessary, for many operations. One such operation is dynamic memory allocation. When you are writing a program that will need to work with an unknown amount of data, dynamic memory allocation allows you to create variables, arrays, and more complex data structures in memory while the program is running. We will discuss dynamic memory allocation in greater detail in this chapter. Pointers are also very useful in algorithms that manipulate arrays and work with certain types of strings. In object-oriented programming, which you will learn about in Chapters 13, 14, and 15, pointers are very useful for creating and working with objects, and for sharing access to those objects.

Creating and Using Pointer Variables

The definition of a pointer variable looks pretty much like any other definition. Here is an example:

```
int *ptr;
```

The asterisk in front of the variable name indicates that `ptr` is a pointer variable. The `int` data type indicates that `ptr` can be used to hold the address of an integer variable. The definition statement above would read "`ptr` is a pointer to an `int`."



NOTE: In this definition, the word `int` does not mean `ptr` is an integer variable. It means `ptr` can hold the address of an integer variable. Remember, pointers only hold one kind of value: an address.

Some programmers prefer to define pointers with the asterisk next to the type name, rather than the variable name. For example, the previous definition shown above could be written as:

```
int* ptr;
```

This style of definition might visually reinforce the fact that `ptr`'s data type is not `int`, but pointer-to-`int`. Both definition styles are correct.



It is never a good idea to define a pointer variable without initializing it with a valid memory address. If you inadvertently use an uninitialized pointer variable, you will be affecting some unknown location in memory. For that reason, it is a good idea to initialize pointer variables with the special value `nullptr`.

In C++ 11, the `nullptr` key word was introduced to represent the address 0. So, assigning `nullptr` to a pointer variable makes the variable point to the address 0. When a pointer is set to the address 0, it is referred to as a *null pointer* because it points to "nothing." Here is an example of how you define a pointer variable and initialize it with the value `nullptr`:

```
int *ptr = nullptr;
```



NOTE: If you are using an older compiler that does not support the C++ 11 standard, you should initialize pointers with the integer 0, or the value `NULL`. The value `NULL` is defined in the `<iostream>` header file (as well as other header files) to represent the value 0.

Program 9-2 demonstrates a very simple usage of a pointer: storing and printing the address of another variable.

Program 9-2

```
1  // This program stores the address of a variable in a pointer.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      int x = 25;          // int variable
8      int *ptr = nullptr;  // Pointer variable, can point to an int
9
10     ptr = &x;            // Store the address of x in ptr
11     cout << "The value in x is " << x << endl;
12     cout << "The address of x is " << ptr << endl;
13     return 0;
14 }
```

Program Output

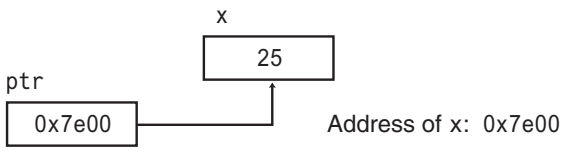
```
The value in x is 25
The address of x is 0x7e00
```

In Program 9-2, two variables are defined: `x` and `ptr`. The variable `x` is an `int`, and the variable `ptr` is a pointer to an `int`. The variable `x` is initialized with the value 25, and `ptr` is initialized with `nullptr`. Then, the variable `ptr` is assigned the address of `x` with the following statement in line 10:

```
ptr = &x;
```

Figure 9-4 illustrates the relationship between `ptr` and `x`.

Figure 9-4 `ptr` points to x



As shown in Figure 9-4, `x`, which is located at memory address 0x7e00, contains the number 25. The `ptr` pointer contains the address 0x7e00. In essence, it "points" to the variable `x`.

The real benefit of pointers is that they allow you to indirectly access and modify the variable being pointed to. In Program 9-2, for instance, `ptr` could be used to change the contents of the variable `x`. This is done with the *indirection operator*, which is an asterisk (*). When the indirection operator is placed in front of a pointer variable name, it *dereferences* the pointer. When you are working with a dereferenced pointer, you are actually working with the value the pointer is pointing to. This is demonstrated in Program 9-3.

Program 9-3

```
1  // This program demonstrates the use of the indirection operator.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      int x = 25;           // int variable
8      int *ptr = nullptr;   // Pointer variable, can point to an int
9
10     ptr = &x;             // Store the address of x in ptr
11
12     // Use both x and ptr to display the value in x.
13     cout << "Here is the value in x, printed twice:\n";
14     cout << x << endl;    // Displays the contents of x
15     cout << *ptr << endl; // Displays the contents of x
16
17     // Assign 100 to the location pointed to by ptr. This
18     // will actually assign 100 to x.
19     *ptr = 100;
```

(program continues)

Program 9-3 *(continued)*

```
20
21      // Use both x and ptr to display the value in x.
22      cout << "Once again, here is the value in x:\n";
23      cout << x << endl;     // Displays the contents of x
24      cout << *ptr << endl; // Displays the contents of x
25      return 0;
26  }
```

Program Output

```
Here is the value in x, printed twice:
25
25
Once again, here is the value in x:
100
100
```

Take a closer look at the statement in line 10:

```
ptr = &x;
```

This statement assigns the address of the `x` variable to the `ptr` variable. Now look at line 15:

```
cout << *ptr << endl; // Displays the contents of x
```

When you apply the indirection operator (`*`) to a pointer variable, you are working not with the pointer variable itself, but with the item it points to. Because this statement sends the expression `*ptr` to the `cout` object, it does not display the value in `ptr`, but the value that `ptr` points to. Since `ptr` points to the `x` variable, this statement displays the contents of the `x` variable.

Suppose the statement did not use the indirection operator. Suppose that statement had been written as:

```
cout << ptr << endl;  // Displays an address
```

Because the indirection operator is not applied to `ptr` in this statement, it works directly with the `ptr` variable. This statement would display the address that is stored in `ptr`.

Now take a look at the following statement, which appears in line 19:

```
*ptr = 100;
```

Notice the indirection operator being used with `ptr`. That means the statement is not affecting `ptr`, but the item that `ptr` points to. This statement assigns 100 to the item `ptr` points to, which is the `x` variable. After this statement executes, 100 will be stored in the `x` variable.

Program 9-4 demonstrates that pointers can point to different variables.

Program 9-4

```
1  // This program demonstrates a pointer variable referencing
2  // different variables.
3  #include <iostream>
4  using namespace std;
```

```
 5
 6  int main()
 7  {
 8      int x = 25, y = 50, z = 75;   // Three int variables
 9      int *ptr = nullptr;           // Pointer variable
10
11      // Display the contents of x, y, and z.
12      cout << "Here are the values of x, y, and z:\n";
13      cout << x << " " << y << " " << z << endl;
14
15      // Use the pointer to manipulate x, y, and z.
16
17      ptr = &x;       // Store the address of x in ptr.
18      *ptr += 100;    // Add 100 to the value in x.
19
20      ptr = &y;       // Store the address of y in ptr.
21      *ptr += 100;    // Add 100 to the value in y.
22
23      ptr = &z;       // Store the address of z in ptr.
24      *ptr += 100;    // Add 100 to the value in z.
25
26      // Display the contents of x, y, and z.
27      cout << "Once again, here are the values of x, y, and z:\n";
28      cout << x << " " << y << " " << z << endl;
29      return 0;
30  }
```

Program Output

```
Here are the values of x, y, and z:
25  50  75
Once again, here are the values of x, y, and z:
125 150 175
```

Take a closer look at the statement in line 17:

```
ptr = &x;
```

This statement assigns the address of the `x` variable to the `ptr` variable. Now look at line 18:

```
*ptr += 100;
```

In this statement, notice the indirection operator (`*`) is used with the `ptr` variable. When we apply the indirection operator to `ptr`, we are working not with `ptr`, but with the item that `ptr` points to. When this statement executes, `ptr` is pointing at `x`, so the statement in line 18 adds 100 to the contents of `x`. Then the following statement, in line 20, executes:

```
ptr = &y;
```

This statement assigns the address of the `y` variable to the `ptr` variable. After this statement executes, `ptr` is no longer pointing at `x`. Rather, it will be pointing at `y`. The statement in line 21, shown here, adds 100 to the `y` variable:

```
*ptr += 100;
```

These steps are repeated with the `z` variable in lines 23 and 24.



NOTE: So far, you've seen three different uses of the asterisk in C++:

- As the multiplication operator, in statements such as

```
distance = speed * time;
```

- In the definition of a pointer variable, such as

```
int *ptr = nullptr;
```

- As the indirection operator, in statements such as

```
*ptr = 100;
```

9.3 The Relationship between Arrays and Pointers

CONCEPT: **Array names can be used as constant pointers, and pointers can be used as array names.**

You learned in Chapter 7 that an array name, without brackets and a subscript, actually represents the starting address of the array. This means that an array name is really a pointer. Program 9-5 illustrates this by showing an array name being used with the indirection operator.

Program 9-5

```
1  // This program shows an array name being dereferenced with the *
2  // operator.
3  #include <iostream>
4  using namespace std;
5
6  int main()
7  {
8      short numbers[] = {10, 20, 30, 40, 50};
9
10     cout << "The first element of the array is ";
11     cout << *numbers << endl;
12     return 0;
13 }
```

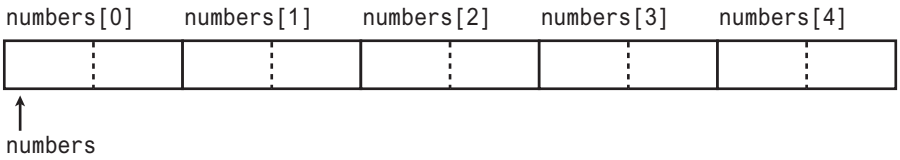
Program Output

```
The first element of the array is 10
```

Because `numbers` works like a pointer to the starting address of the array, the first element is retrieved when `numbers` is dereferenced. So how could the entire contents of an array be retrieved using the indirection operator? Remember, array elements are stored together in memory, as illustrated in Figure 9-5.

It makes sense that if `numbers` is the address of `numbers[0]`, values could be added to `numbers` to get the addresses of the other elements in the array. It's important to know, however, that pointers do not work like regular variables when used in mathematical statements. In C++, when you add a value to a pointer, you are actually adding that value *times*

Figure 9-5 Array elements are stored in contiguous memory locations



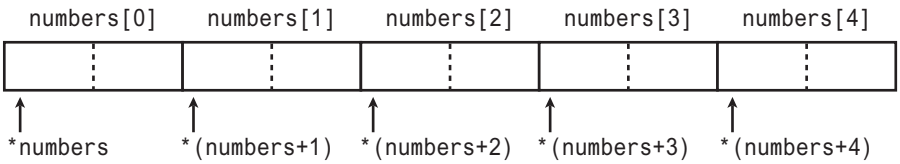
the size of the data type being referenced by the pointer. In other words, if you add one to `numbers`, you are actually adding `1 * sizeof(short)` to `numbers`. If you add 2 to `numbers`, the result is `numbers + 2 * sizeof(short)`, and so forth. On a typical system, this means the following are true, because `short` integers typically use 2 bytes:

```
*(numbers + 1) is actually *(numbers + 1 * 2)
*(numbers + 2) is actually *(numbers + 2 * 2)
*(numbers + 3) is actually *(numbers + 3 * 2)
```

and so forth.

This automatic conversion means an element in an array can be retrieved by using its subscript, or by adding its subscript to a pointer to the array. If the expression `*numbers`, which is the same as `*(numbers + 0)`, retrieves the first element in the array, then `*(numbers + 1)` retrieves the second element. Likewise, `*(numbers + 2)` retrieves the third element, and so forth. Figure 9-6 shows the equivalence of subscript notation and pointer notation.

Figure 9-6 Subscript notation and pointer notation





NOTE: The parentheses are critical when adding values to pointers. The `*` operator has precedence over the `+` operator, so the expression `*number + 1` is not equivalent to `*(number + 1)`. `*number + 1` adds one to the contents of the first element of the array, while `*(number + 1)` adds one to the address in `number`, then dereferences it.

Program 9-6 shows the entire contents of the array being accessed, using pointer notation.

Program 9-6

```
1  // This program processes an array using pointer notation.
2  #include <iostream>
3  using namespace std;
4
```

(program continues)

Program 9-6 *(continued)*

```
 5  int main()
 6  {
 7       const int SIZE = 5; // Size of the array
 8       int numbers[SIZE];  // Array of integers
 9       int count;          // Counter variable
10
11       // Get values to store in the array.
12       // Use pointer notation instead of subscripts.
13       cout << "Enter " << SIZE << " numbers: ";
14       for (count = 0; count < SIZE; count++)
15            cin >> *(numbers + count);
16
17       // Display the values in the array.
18       // Use pointer notation instead of subscripts.
19       cout << "Here are the numbers you entered:\n";
20       for (count = 0; count < SIZE; count++)
21            cout << *(numbers + count)<< " ";
22       cout << endl;
23       return 0;
24  }
```

Program Output with Example Input Shown in Bold

```
Enter 5 numbers: 5 10 15 20 25 Enter
Here are the numbers you entered:
5 10 15 20 25
```

When working with arrays, remember the following rule:

array`[index]` is equivalent to `*(`*array* `+ index)`



WARNING! Remember that C++ performs no bounds checking with arrays. When stepping through an array with a pointer, it's possible to give the pointer an address outside of the array.

To demonstrate just how close the relationship is between array names and pointers, look at Program 9-7. It defines an array of `double`s and a `double` pointer, which is assigned the starting address of the array. Not only is pointer notation then used with the array name, but subscript notation is also used with the pointer!

Program 9-7

```
1  // This program uses subscript notation with a pointer variable and
2  // pointer notation with an array name.
3  #include <iostream>
4  using namespace std;
5
6  int main()
```

```
 7  {
 8     const int NUM_COINS = 5;
 9     double coins[NUM_COINS] = {0.05, 0.1, 0.25, 0.5, 1.0};
10     double *doublePtr;    // Pointer to a double
11     int count;            // Array index
12
13     // Assign the address of the coins array to doublePtr.
14     doublePtr = coins;
15
16     // Display the contents of the coins array. Use subscripts
17     // with the pointer!
18     cout << "Here are the values in the coins array:\n";
19     for (count = 0; count < NUM_COINS; count++)
20        cout << doublePtr[count] << " ";
21
22     // Display the contents of the array again, but this time
23     // use pointer notation with the array name!
24     cout << "\nAnd here they are again:\n";
25     for (count = 0; count < NUM_COINS; count++)
26        cout << *(coins + count) << " ";
27     cout << endl;
28     return 0;
29  }
```

Program Output

```
Here are the values in the coins array:
0.05 0.1 0.25 0.5 1
And here they are again:
0.05 0.1 0.25 0.5 1
```

Notice the address operator is not needed when an array's address is assigned to a pointer. Because the name of an array is already an address, use of the & operator would be incorrect. You can, however, use the address operator to get the address of an individual element in an array. For instance, `&numbers[1]` gets the address of `numbers[1]`. This technique is used in Program 9-8.

Program 9-8

```
 1  // This program uses the address of each element in the array.
 2  #include <iostream>
 3  using namespace std;
 4
 5  int main()
 6  {
 7      const int NUM_COINS = 5;
 8      double coins[NUM_COINS] = {0.05, 0.1, 0.25, 0.5, 1.0};
 9      double *doublePtr = nullptr;      // Pointer to a double
10      int count;                        // Array index
11
```

(program continues)

Program 9-8 *(continued)*

```
12      // Use the pointer to display the values in the array.
13      cout << "Here are the values in the coins array:\n";
14      for (count = 0; count < NUM_COINS; count++)
15      {
16         // Get the address of an array element.
17         doublePtr = &coins[count];
18
19         // Display the contents of the element.
20         cout << *doublePtr << " ";
21      }
22      cout << endl;
23      return 0;
24  }
```

Program Output

```
Here are the values in the coins array:
0.05 0.1 0.25 0.5 1
```

The only difference between array names and pointer variables is that you cannot change the address an array name points to. For example, consider the following definitions:

```
double readings[20], totals[20];
double *dptr = nullptr;
```

These statements are legal:

```
dptr = readings;  // Make dptr point to readings.
dptr = totals;    // Make dptr point to totals.
```

But these are illegal:

```
readings = totals;  // ILLEGAL! Cannot change readings.
totals = dptr;      // ILLEGAL! Cannot change totals.
```

Array names are *pointer constants*. You can't make them point to anything but the array they represent.

9.4 Pointer Arithmetic

CONCEPT: Some mathematical operations may be performed on pointers.

The contents of pointer variables may be changed with mathematical statements that perform addition or subtraction. This is demonstrated in Program 9-9. The first loop increments the pointer variable, stepping it through each element of the array. The second loop decrements the pointer, stepping it backwards through the array.

Program 9-9

```
1  // This program uses a pointer to display the contents of an array.
2  #include <iostream>
3  using namespace std;
4
5  int main()
6  {
7      const int SIZE = 8;
8      int set[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
9      int *numPtr = nullptr;  // Pointer
10     int count;              // Counter variable for loops
11
12     // Make numPtr point to the set array.
13     numPtr = set;
14
15     // Use the pointer to display the array contents.
16     cout << "The numbers in set are:\n";
17     for (count = 0; count < SIZE; count++)
18     {
19         cout << *numPtr << " ";
20         numPtr++;
21     }
22
23     // Display the array contents in reverse order.
24     cout << "\nThe numbers in set backward are:\n";
25     for (count = 0; count < SIZE; count++)
26     {
27         numPtr--;
28         cout << *numPtr << " ";
29     }
30     return 0;
31 }
```

Program Output

```
The numbers in set are:
5 10 15 20 25 30 35 40
The numbers in set backward are:
40 35 30 25 20 15 10 5
```



NOTE: Because `numPtr` is a pointer to an integer, the increment operator adds the size of one integer to `numPtr`, so it points to the next element in the array. Likewise, the decrement operator subtracts the size of one integer from the pointer.

Not all arithmetic operations may be performed on pointers. For example, you cannot multiply or divide a pointer. The following operations are allowable:

- The `++` and `--` operators may be used to increment or decrement a pointer variable.
- An integer may be added to or subtracted from a pointer variable. This may be performed with the `+` and `-` operators, or the `+=` and `-=` operators.
- A pointer may be subtracted from another pointer.

9.5 Initializing Pointers

CONCEPT: Pointers may be initialized with the address of an existing object.

Remember a pointer is designed to point to an object of a specific data type. When a pointer is initialized with an address, it must be the address of an object the pointer can point to. For instance, the following definition of `pint` is legal because `myValue` is an integer:

```
int myValue;
int *pint = &myValue;
```

The following is also legal because `ages` is an array of integers:

```
int ages[20];
int *pint = ages;
```

But the following definition of `pint` is illegal because `myFloat` is not an `int`:

```
float myFloat;
int *pint = &myFloat;  // Illegal!
```

Pointers may be defined in the same statement as other variables of the same type. The following statement defines an integer variable, `myValue`, then defines a pointer, `pint`, which is initialized with the address of `myValue`:

```
int myValue, *pint = &myValue;
```

And the following statement defines an array, `readings`, and a pointer, `marker`, which is initialized with the address of the first element in the array:

```
double readings[50], *marker = readings;
```

Of course, a pointer can only be initialized with the address of an object that has already been defined. The following is illegal, because `pint` is being initialized with the address of an object that does not exist yet:

```
int *pint = &myValue;  // Illegal!
int myValue;
```

Checkpoint

9.1 Write a statement that displays the address of the variable `count`.

9.2 Write the definition statement for a variable `fltPtr`. The variable should be a pointer to a `float`.

9.3 List three uses of the * symbol in C++.

9.4 What is the output of the following code?

```
int x = 50, y = 60, z = 70;
int *ptr = nullptr;

cout << x << " " << y << " " << z << endl;
ptr = &x;
```

```
*ptr *= 10;
ptr = &y;
*ptr *= 5;
ptr = &z;
*ptr *= 2;
cout << x << " " << y << " " << z << endl;
```

9.5 Rewrite the following loop so it uses pointer notation (with the indirection operator) instead of subscript notation.

```
for (int x = 0; x < 100; x++)
   cout << arr[x] << endl;
```

9.6 Assume `ptr` is a pointer to an `int` and holds the address 12000. On a system with 4-byte integers, what address will be in `ptr` after the following statement?

```
ptr += 10;
```

9.7 Assume `pint` is a pointer variable. Is each of the following statements valid or invalid? If any is invalid, why?

A) `pint++;`
B) `--pint;`
C) `pint /= 2;`
D) `pint *= 4;`
E) `pint += x; // Assume x is an int.`

9.8 Is each of the following definitions valid or invalid? If any is invalid, why?

A)
```
int ivar;
int *iptr = &ivar;
```
B) `int ivar, *iptr = &ivar;`
C)
```
float fvar;
int *iptr = &fvar;
```
D) `int nums[50], *iptr = nums;`
E)
```
int *iptr = &ivar;
int ivar;
```

9.6 Comparing Pointers

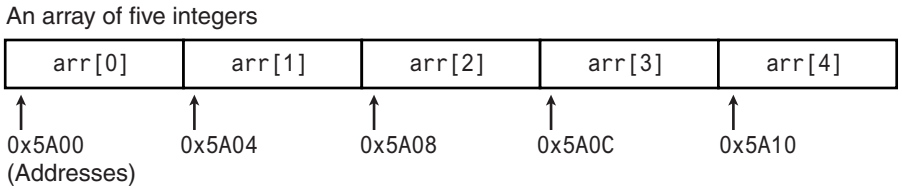
CONCEPT: **If one address comes before another address in memory, the first address is considered "less than" the second. C++'s relational operators may be used to compare pointer values.**

Pointers may be compared by using any of C++'s relational operators:

```
>> < == != >= <=
```

In an array, all the elements are stored in consecutive memory locations, so the address of element 1 is greater than the address of element 0. This is illustrated in Figure 9-7.

Figure 9-7 Array elements



Because the addresses grow larger for each subsequent element in the array, the expressions tested by the following `if` statements are all true:

```
if (&arr[1] > &arr[0])
if (arr < &arr[4])
if (arr == &arr[0])
if (&arr[2] != &arr[3])
```



NOTE: Comparing two pointers is not the same as comparing the values the two pointers point to. For example, the following `if` statement compares the addresses stored in the pointer variables `ptr1` and `ptr2`:

```
if (ptr1 < ptr2)
```

The following statement, however, compares the values that `ptr1` and `ptr2` point to:

```
if (*ptr1 < *ptr2)
```

The capability of comparing addresses gives you another way to be sure a pointer does not go beyond the boundaries of an array. Program 9-10 initializes the pointer `ptr` with the starting address of the array `numbers`. The `ptr` pointer is then stepped through the `numbers` array until the address it contains is equal to the address of the last element of the array. Then, the pointer is stepped backward through the array until it points to the first element.

Program 9-10

```
1  // This program uses a pointer to display the contents
2  // of an integer array.
3  #include <iostream>
4  using namespace std;
5
6  int main()
7  {
8      const int SIZE = 8;
9      int numbers[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
10     int *ptr = numbers;      // Make ptr point to numbers
11
12     // Display the numbers in the array.
13     cout << "The numbers are:\n";
14     cout << *ptr << " ";     // Display first element
```

```
15      while (ptr < &numbers[SIZE - 1])
16      {
17         // Advance ptr to point to the next element.
18         ptr++;
19         // Display the value pointed to by ptr.
20         cout << *ptr << " ";
21      }
22
23      // Display the numbers in reverse order.
24      cout << "\nThe numbers backward are:\n";
25      cout << *ptr << " ";    // Display first element
26      while (ptr > numbers)
27      {
28         // Move backward to the previous element.
29         ptr--;
30         // Display the value pointed to by ptr.
31         cout << *ptr << " ";
32      }
33      return 0;
34   }
```

Program Output

```
The numbers are:
5 10 15 20 25 30 35 40
The numbers backward are:
40 35 30 25 20 15 10 5
```

9.7 Pointers as Function Parameters

CONCEPT: **A pointer can be used as a function parameter. It gives the function access to the original argument, much like a reference parameter does.**

In Chapter 6, you were introduced to the concept of reference variables being used as function parameters. A reference variable acts as an alias to the original variable used as an argument. This gives the function access to the original argument variable, allowing it to change the variable's contents. When a variable is passed into a reference parameter, the argument is said to be passed by reference.

Another way to pass an argument by reference is to use a pointer variable as the parameter. Admittedly, reference variables are much easier to work with than pointers. Reference variables hide all the "mechanics" of dereferencing and indirection. You should still learn to use pointers as function arguments, however, because some tasks (especially when you are dealing with strings) are best done with pointers.* Also, the C++ library has many functions that use pointers as parameters.

*It is also important to learn this technique in case you ever need to write a C program. In C, the only way to pass a variable by reference is to use a pointer.

Here is the definition of a function that uses a pointer parameter:

```
void doubleValue(int *val)
{
   *val *= 2;
}
```

The purpose of this function is to double the variable pointed to by `val` with the following statement:

```
*val *= 2;
```

When `val` is dereferenced, the `*=` operator works on the variable pointed to by `val`. This statement multiplies the original variable, whose address is stored in `val`, by 2. Of course, when the function is called, the address of the variable that is to be doubled must be used as the argument, not the variable itself. Here is an example of a call to the `doubleValue` function:

```
doubleValue(&number);
```

This statement uses the address operator (`&`) to pass the address of `number` into the `val` parameter. After the function executes, the contents of `number` will have been multiplied by 2. The use of this function is illustrated in Program 9-11.

Program 9-11

```
1  // This program uses two functions that accept addresses of
2  // variables as arguments.
3  #include <iostream>
4  using namespace std;
5
6  // Function prototypes
7  void getNumber(int *);
8  void doubleValue(int *);
9
10 int main()
11 {
12     int number;
13
14     // Call getNumber and pass the address of number.
15     getNumber(&number);
16
17     // Call doubleValue and pass the address of number.
18     doubleValue(&number);
19
20     // Display the value in number.
21     cout << "That value doubled is " << number << endl;
22     return 0;
23 }
24
```

```
25  //*************************************************************
26  // Definition of getNumber. The parameter, input, is a pointer. *
27  // This function asks the user for a number. The value entered  *
28  // is stored in the variable pointed to by input.               *
29  //*************************************************************
30
31  void getNumber(int *input)
32  {
33      cout << "Enter an integer number: ";
34      cin >> *input;
35  }
36
37  //*************************************************************
38  // Definition of doubleValue. The parameter, val, is a pointer. *
39  // This function multiplies the variable pointed to by val by   *
40  // two.                                                         *
41  //*************************************************************
42
43  void doubleValue(int *val)
44  {
45      *val *= 2;
46  }
```

Program Output with Example Input Shown in Bold

```
Enter an integer number: 10 Enter
That value doubled is 20
```

Program 9-11 has two functions that use pointers as parameters. Notice the function prototypes:

```
void getNumber(int *);
void doubleValue(int *);
```

Each one uses the notation `int *` to indicate the parameter is a pointer to an `int`. As with all other types of parameters, it isn't necessary to specify the name of the variable in the prototype. The `*` is required, though.

The `getNumber` function asks the user to enter an integer value. The following `cin` statement, in line 34, stores the value entered by the user in memory:

```
cin >> *input;
```

The indirection operator causes the value entered by the user to be stored, not in `input`, but in the variable pointed to by `input`.



WARNING! It's critical that the indirection operator be used in the statement above. Without it, `cin` would store the value entered by the user in `input`, as if the value were an address. If this happens, `input` will no longer point to the `number` variable in function `main`. Subsequent use of the pointer will result in erroneous, if not disastrous, results.

When the `getNumber` function is called in line 15, the address of the `number` variable in function `main` is passed as the argument. After the function executes, the value entered by the user is stored in `number`. Next, the `doubleValue` function is called in line 18, with the address of `number` passed as the argument. This causes `number` to be multiplied by 2.

Pointer variables can also be used to accept array addresses as arguments. Either subscript or pointer notation may then be used to work with the contents of the array. This is demonstrated in Program 9-12.

Program 9-12

```
1  // This program demonstrates that a pointer may be used as a
2  // parameter to accept the address of an array.
3  #include <iostream>
4  #include <iomanip>
5  using namespace std;
6
7  // Function prototypes
8  void getSales(double *, int);
9  double totalSales(double *, int);
10
11 int main()
12 {
13     const int QTRS = 4;
14     double sales[QTRS];
15
16     // Get the sales data for all quarters.
17     getSales(sales, QTRS);
18
19     // Set the numeric output formatting.
20     cout << fixed << showpoint << setprecision(2);
21
22     // Display the total sales for the year.
23     cout << "The total sales for the year are $";
24     cout << totalSales(sales, QTRS) << endl;
25     return 0;
26 }
27
28 //*************************************************************
29 // Definition of getSales. This function uses a pointer to accept *
30 // the address of an array of doubles. The function asks the user *
31 // to enter sales figures and stores them in the array.          *
32 //*************************************************************
33 void getSales(double *arr, int size)
34 {
35     for (int count = 0; count < size; count++)
36     {
37         cout << "Enter the sales figure for quarter ";
38         cout << (count + 1) << ": ";
39         cin >> arr[count];
40     }
41 }
42
```

```
43  //*************************************************************
44  // Definition of totalSales. This function uses a pointer to      *
45  // accept the address of an array. The function returns the total *
46  // of the elements in the array.                                  *
47  //*************************************************************
48  double totalSales(double *arr, int size)
49  {
50       double sum = 0.0;
51
52       for (int count = 0; count < size; count++)
53       {
54            sum += *arr;
55            arr++;
56       }
57       return sum;
58  }
```

Program Output with Example Input Shown in Bold

```
Enter the sales figure for quarter 1: 10263.98 Enter
Enter the sales figure for quarter 2: 12369.69 Enter
Enter the sales figure for quarter 3: 11542.13 Enter
Enter the sales figure for quarter 4: 14792.06 Enter
The total sales for the year are $48967.86
```

Notice in the `getSales` function in Program 9-12, even though the parameter `arr` is defined as a pointer, subscript notation is used in the `cin` statement in line 39:

```
cin >> arr[count];
```

In the `totalSales` function, `arr` is used with the indirection operator in line 54:

```
sum += *arr;
```

And in line 55, the address in `arr` is incremented to point to the next element:

```
arr++;
```



NOTE: The two previous statements could be combined into the following statement:

```
sum += *arr++;
```

The `*` operator will first dereference `arr`, then the `++` operator will increment the address in `arr`.

Pointers to Constants

You have seen how an item's address can be passed into a pointer parameter, and how the pointer can be used to modify the item that was passed as an argument. Sometimes it is necessary to pass the address of a `const` item into a pointer. When this is the case, the pointer must be defined as a pointer to a `const` item. For example, consider the following array definition:

```
const int SIZE = 6;
const double payRates[SIZE] = { 18.55, 17.45,
                                12.85, 14.97,
                                10.35, 18.89 };
```

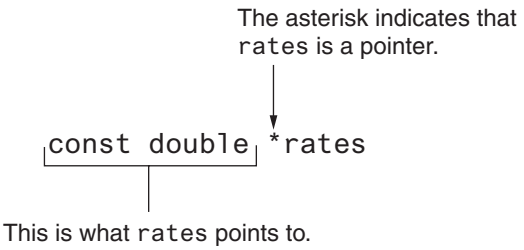
In this code, `payRates` is an array of `const doubles`. This means each element in the array is a `const double`, and the compiler will not allow us to write code that changes the array's contents. If we want to pass the `payRates` array into a pointer parameter, the parameter must be declared as a pointer to `const double`. The following function shows such an example:

```
void displayPayRates(const double *rates, int size)
{
     // Set numeric output formatting.
     cout << setprecision(2) << fixed << showpoint;

     // Display all the pay rates.
     for (int count = 0; count < size; count++)
     {
          cout << "Pay rate for employee " << (count + 1)
               << " is $" << *(rates + count) << endl;
     }
}
```

In the function header, notice the `rates` parameter is defined as a pointer to `const double`. It should be noted the word `const` is applied to the thing that `rates` points to, not `rates` itself. This is illustrated in Figure 9-8.

Figure 9-8 `rates` is a pointer to `const double`



Because `rates` is a pointer to a `const`, the compiler will not allow us to write code that changes the thing that `rates` points to.

In passing the address of a constant into a pointer variable, the variable must be defined as a pointer to a constant. If the word `const` had been left out of the definition of the `rates` parameter, a compiler error would have resulted.

Passing a Nonconstant Argument into a Pointer to a Constant

Although a constant's address can be passed only to a pointer to `const`, a pointer to `const` can also receive the address of a nonconstant item. For example, look at Program 9-13.